

MEDIATEK

MT8676 Android SPI

Version: 1.1
Release date: 2025-03-24

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Jia Xue	Official release
1.1	2025-03-24	Alex Gan	Added Section 1.3.2 Node Configuration

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 SPI 4

1.1 Overview 4

1.1.1 Introduction 4

1.1.2 Abbreviation 4

1.2 Architecture/Process Overview 4

1.2.1 Introduction to SPI 4

1.2.2 MT8676 SPI Characteristics 4

1.2.3 SPI Transmission Format 6

1.3 Configuration/Customization Guideline 8

1.3.1 Device Tree 8

1.3.2 Node Configuration 8

1.3.3 PAD_SEL 9

1.3.4 Frequency 9

1.3.5 SPI Mode 9

1.3.6 FIFO and DMA Mode 10

1.3.7 Support for Multiple Devices 10

1.3.8 Testing and Debugging 11

1.4 Frequently Asked Questions/Troubleshooting 11

1.4.1 SPI Issue Debugging Methods 11

1.4.2 DTS Configuration 11

1.4.3 Confirm GPIO Mode 12

1.4.4 Measuring Waveforms 12

1.4.5 How to Print SPI Register Information 12

1.4.6 Seeking MTK Support 13

Exhibit 1 Terms and Conditions 14

List of Figures

Figure 1-1. Pin connection between SPI master and SPI slave 4

Figure 1-2. Waveforms of four communication modes 5

Figure 1-3. Master device connected to multiple device pins 5

Figure 1-4. SPI transmission format 6

List of Tables

Table 1-1. Abbreviations 4

1 SPI

1.1 Overview

1.1.1 Introduction

This chapter introduces the hardware, software and functions of the MT8676 SPI controller.

1.1.2 Abbreviation

Table 1-1. Abbreviations

Abbreviation	Explanation
CPHA	Clock Phase
CPOL	Clock Polarity
CS	Chip Select Pin
DMA	Direct Memory Access
FIFO	First In First Out
MISO	Master In Slave Out
MOSI	Master Out Slave In
PIO	Programmed Input/Output Model
SCLK	SPI Clock
SPI	Serial Peripheral Interface

1.2 Architecture/Process Overview

1.2.1 Introduction to SPI

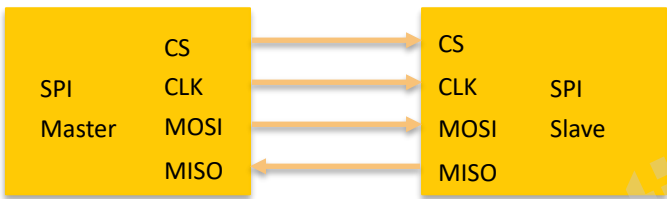


Figure 1-1. Pin connection between SPI master and SPI slave

SPI interface is a bit-serial, four-pin transmission protocol. Figure 1-1 is an example of the connection between SPI master and SPI slave. SPI controller interface is a master responsible of the data transmission with the slave.

1.2.2 MT8676 SPI Characteristics

- Provides eight SPI ports
- Supports two transmission modes: DMA and FIFO

- FIFO mode supports up to 32 bytes in a single transmission
- If the transmission length is less than 1024 bytes, DMA mode supports up to 1024 bytes
- DMA mode supports multiples of 1024 bytes (length = number of cycles * 1024, where $1 \leq \text{number of cycles} \leq 256$).
- The maximum transmission frequency is 52 MHz
- There are four communication modes available (Mode 0, 1, 2, 3), as shown in Figure 1-2.
- This essentially defines the SCLK edge on which the MOSI line switches, the SCLK edge on which the master samples
- the MISO line, and the stable level of the SCLK signal (e.g., the clock level when the clock is not active, either high or low). Each mode is defined by a pair of parameters called “Clock Polarity” (CPOL) and “Clock Phase” (CPHA).

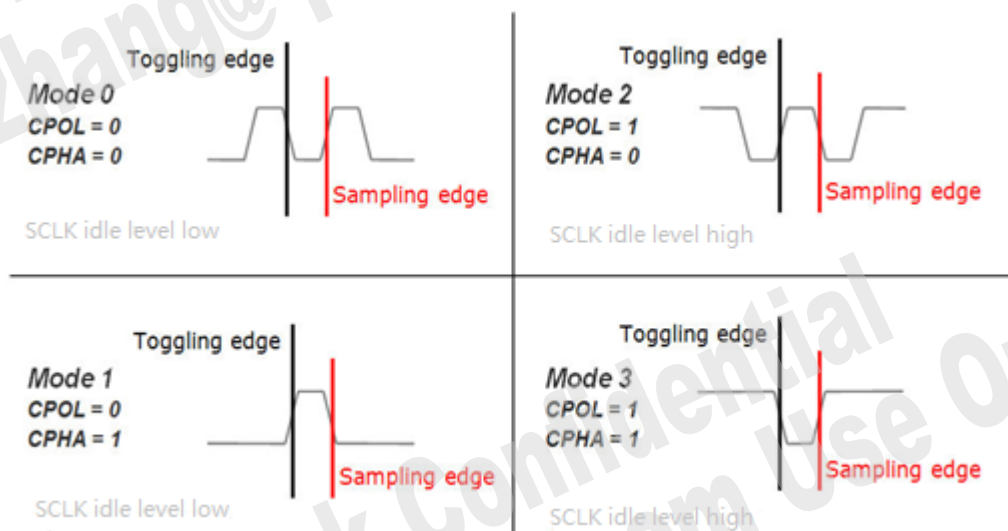


Figure 1-2. Waveforms of four communication modes

- SPI controller has only one CS pin. That is to say, it can only support a single slave. However, you can use GPIO as CS to support multiple devices, as shown in Figure 1-3.

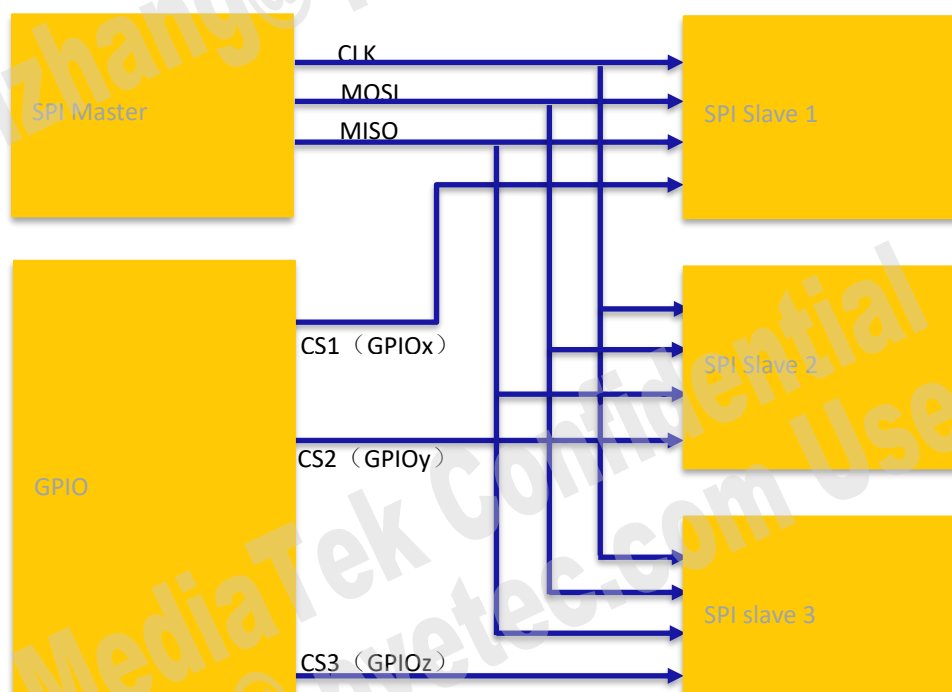


Figure 1-3. Master device connected to multiple device pins

1.2.3 SPI Transmission Format

In DMA mode, the data to be transmitted should be prepared in advance in the system. In PIO mode, the system should first push the data to be transmitted into the SPI TX FIFO. Upon receiving the START command, the SPI will continuously send data to the slave while receiving data from the slave.

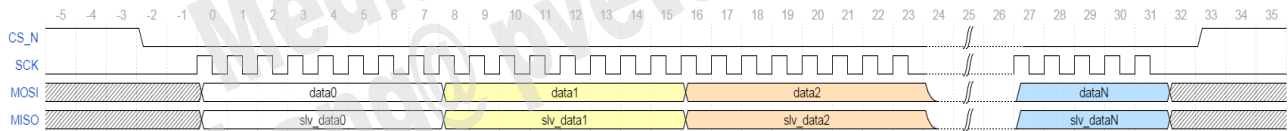


Figure 1-4. SPI transmission format

1.2.3.1 Interface Description

1.2.3.2 spi_sync

Prototype	Parameter	Return Value
int spi_sync (struct spi_device *spi, struct spi_message *message)	SPI: Pointer to the structure spi_device	Return 0 if successful, otherwise return an error code
	Message: Pointer to the structure spi_message	

1.2.3.3 spi_async

Prototype	Parameter	Return Value
int spi_async (struct spi_device *spi, struct spi_message *message)	SPI: Pointer to the structure spi_device	Return 0 if successful, otherwise return an error code
	Message: Pointer to the structure spi_message	

1.2.3.4 spi_write_then_read

The parameters for this routine always use a small buffer for copying and should not be used for more than 32 bytes. Performance-sensitive or bulk transfer code should instead use spi_{async, sync}() calls with DMA-safe buffers.

Prototype	Parameter	Return Value
int spi_write_then_read (struct spi_device *spi, const void *txbuf, unsigned n_tx, unsigned n_tx, void *rxbuf, unsigned n_rx)	SPI: Pointer to the structure spi_device	Return 0 if successful, otherwise return an error code
	txbuf: Data to be written	
	n_tx: Size of txbuf (in bytes)	
	Rxbuf: Buffer where data will be read into	
	n_rx: Size of rxbuf (in bytes)	

1.2.3.5 Programming Guide

1.2.3.6 Data Transfer with spi_sync

Data is exchanged between SPI controller and SPI slave devices, and thus the steps for sending data are the same as those for receiving data. For example, sending and receiving data.

```
struct spi_message msg;
struct spi_transfer xfer;

spi_message_init(&msg);
memset(&xfer, 0, sizeof(xfer));

xfer.tx_buf = local_tx_buf;
xfer.rx_buf = local_rx_buf;
xfer.len = data_length;
spi_message_add_tail(&xfer, &msg);

ret = spi_sync(spi, &msg);
if (ret) {
    .....;
}
```

1.2.3.1 Data Transfer with spi_async

spi_sync function is synchronous. If you wish to use an asynchronous function, you should use spi_async instead.

Please note that the callback function should be registered to spi_message.complete. Source code path:

drivers/spi/spi.c.

spi_message.complete callback function:

```
static void xxxxxx_complete(void *args)
{
    //transfer was success.....;
}
```

Sending and receiving data in asynchronous:

```
struct spi_message msg;
struct spi_transfer xfer;

spi_message_init(&msg);
memset(&xfer, 0, sizeof(xfer));

xfer.tx_buf = local_tx_buf;
xfer.rx_buf = local_rx_buf;
xfer.len = data_length;
spi_message_add_tail(&xfer, &msg);

msg.complete = xxxxxx_complete;
msg.context = args;
```

```
ret = spi_async(spi, &msg);
if(ret) {
    .....;
}
```

1.3 Configuration/Customization Guideline

1.3.1 Device Tree

Source code path: kernel-xxx/arch/arm64/boot/dts/mediatek/

- Add pinctrl to set SPI pinmux

```
&pio {
    spi_pins: spi@0 {
        pins_spi {
            pinmux = <PINMUX_GPIOxxx_FUNC_SPIM1_CSB>,
                < PINMUX_GPIOxxx_FUNC_SPIM1_CLK >,
                < PINMUX_GPIOxxx_FUNC_SPIM1_MO >,
                < PINMUX_GPIOxxx_FUNC_SPIM1_MI >;
            bias-disable;
        };
    };
};
```

- Add slave device on the SPI bus

```
&spi {
    pinctrl-names = "default";
    pinctrl-0 = <&spi_pins>;
    status = "okay";
    spidev0: spi@0 {
        compatible = "mediatek,spi-mt65xx-test";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};
```

1.3.2 Node Configuration

In the device tree, the SPI node is in the “disabled” state by default: status = "disabled":

```
spi: spi@xxxxxxxx {
    compatible = "mediatek,mtxxxx-spi";
    mediatek,pad-select = <0>;
    .....
    .....
    status = "disabled";//Default disabled
};
```

When in use, configure it in the corresponding project DTS: `status = "okay"`:

```
&spi {
    pinctrl-names = "default";
    pinctrl-0 = <&spi_pins>;
    mediatek,pad-select = <0>;
    status = "okay"; // Manually configure to enable
    spidev0: spi@0 {
        compatible = "mediatek,spi-mt65xx-test";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};
```

1.3.3 PAD_SEL

On some ICs, a set of SPIs may have different GPIOs available for selection. The specific group of GPIOs used by SPI is determined based on the GPIO table information. For instance, on the MT8675, SPI4 has options SPI4_A, SPI4_B, and SPI4_C. At this point, it is necessary to add `mediatek,pad-select = <x>;` in the dts file.

```
&spi {
    pinctrl-names = "default";
    pinctrl-0 = <&spi_pins>;
    mediatek,pad-select = <0>; // x = 0, 1, 2 correspond to A, B, C, respectively.
    status = "okay";
    spidev0: spi@0 {
        compatible = "mediatek,spi-mt65xx-test";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};
```

1.3.4 Frequency

SPI controller supports even division frequency scaling, with the maximum frequency for the SPI controller being 52 MHz, and $SPI_SCK \leq 52MHz$. You can set the SPI_CLK in two ways:

1. In the device tree, configure the property `spi-max-frequency`, for example, `spi-max-frequency = <1000000>;` to set the default SPI_CLK to 1MHz.
2. In the device driver, configure the `speed_hz` field of the `spi_transfer` structure, for example, `xfer->speed_hz = 1000000;` to set the SPI_CLK to 1MHz, which will take effect for this transfer.

1.3.5 SPI Mode

You can set the SPI mode in two ways:

1. Set the SPI mode in the device tree

```
&spi {
    pinctrl-names = "default";
    pinctrl-0 = <&spi_pins>;
```

```
status = "okay";
spidev0: spi@0 {
    compatible = "mediatek,spi-mt65xx-test";
    reg = <0>;
    spi-cpol = <0>;
    spi-cpha = <0>;
    spi-max-frequency = <1000000>;
};
```

2. Set the SPI mode in the device driver program

```
static int xxxxxx_probe(struct spi_device *spi)
{
    .....;
    spi->mode = SPI_MODE_0; // SPI_MODE_0, SPI_MODE_1, SPI_MODE_2, SPI_MODE_3
    .....;
}
```

1.3.6 FIFO and DMA Mode

SPI controller supports data transmission in DMA mode and FIFO mode. The controller automatically selects the mode based on the length of the data being transferred. FIFO mode is used when the length is less than or equal to 32 bytes, and DMA mode is used when the length is greater than 32 bytes.

1.3.7 Support for Multiple Devices

If you want to add multiple devices on the SPI bus, you can use GPIOs to support SPI_CS. For example, to add two SPI devices on SPI bus 0, use GPIO136 and GPIO30 as SPI_CS1 and SPI_CS2, respectively, as shown below (the GPIO numbers are used only as examples; please refer to the actual numbers in your setup):

```
&pio {
    spi_pins: spi@0 {
        pins_spi {
            pinmux = //<PINMUX_GPIO136_FUNC_SPIM1_CSB>, //SPI native cs
                    < PINMUX_GPIO137_FUNC_SPIM1_CLK >,
                    < PINMUX_GPIO138_FUNC_SPIM1_MO >,
                    < PINMUX_GPIO139_FUNC_SPIM1_MI >;
            bias-disable;
        };
        pins_spi_cs {
            pinmux = < PINMUX_GPIO136_FUNC_SPIM1_CSB >, //SPI_CS1
                    < PINMUX_GPIO30_FUNC_GPIO30 >, //SPI_CS2
            output-high;
        };
    };
};

&spi {
    pinctrl-names = "default";
```

```

pinctrl-0 = <&spi_pins>;
cs-gpios = <&pio 136 0>,<&pio 30 0>;
status = "okay";
spidev0: spi@0 {
    compatible = "xxxxxx";
    reg = <0>;
    spi-max-frequency = <1000000>;
};
spidev1: spi@1 {
    compatible = "xxxxxx";
    reg = <1>;
    spi-max-frequency = <1000000>;
};
};

```

1.3.8 Testing and Debugging

You can follow the steps introduced above to configure SPI and then verify if the data transmission function is working properly, for example, to verify SPI bus 0: Connect SPI_MOSI to SPI_MISO on the hardware platform. Use spi-mt65xx-dev.c as the device driver. Enter the following test commands:

```

echo -w len=32 > /sys/bus/spi/devices/spixxx/spi
echo -w len=1024 > /sys/bus/spi/devices/spixxx/spi

```

Spixxx represents different groups of SPI, subject to what is actually displayed.

1.4 Frequently Asked Questions/Troubleshooting

1.4.1 SPI Issue Debugging Methods

Most SPI issues encountered can first be debugged in the following ways:

- Check if the DTS configuration, GPIO mode, and pad-select properties are correct.
- Verify that the SPI register information is correct.
- Measure the waveform to see if it meets expectations, such as whether the TX data is correctly output and whether the voltage values are normal.
- If receiving data is abnormal and somewhat regular, such as shifting, try lowering the speed or adjusting the value of tick_dly.
- Ensure that the device driver code is standard, such as whether the TX and RX buffers are allocated and whether the TX/RX data lengths are the same.

1.4.2 DTS Configuration

Please check if the DTS configuration is correct based on the information provided earlier.

1.4.3 Confirm GPIO Mode

To view the status of pins in the Kernel, enter the following commands:

```
# cd /sys
# find -name mt_gpio
# cat mt_gpio
```

e.g.,

```
# cat /sys/devices/platform/soc/1000b000.pinctrl/mt_gpio
PIN: [MODE] [DIR] [DOUT] [DIN] [PULL_EN] [PULL_SEL] [IES] [SMT] [DRIVE] ( [R1] [R0] )
0: 0 0 0 0 1 0 1 0 0
1: 0 0 0 0 1 0 1 0 0
2: 0 1 1 1 1 0 1 0 0
3: 6 0 0 0 1 0 1 0 0
```

1.4.4 Measuring Waveforms

Use an oscilloscope to measure if the SPI waveforms meet expectations.

1.4.5 How to Print SPI Register Information

Add the following content in driver/spi/spi-mt65xx.c

```
#define LOG_CLOSE 0
#define LOG_OPEN 1
u8 spi_log_status = LOG_OPEN;

#define spi_debug(fmt, args...) do { \
    if (spi_log_status == LOG_OPEN) { \
        pr_info("[spi]%s() " fmt, __func__, ##args);\
    } \
} while (0)

static void spi_dump_reg(struct mtk_spi *mdata)
{
    spi_debug("||*****%s*****||\n", __func__);
    spi_debug("cfg0:0x%.8x\n", readl(mdata->base + SPI_CFG0_REG));
    spi_debug("cfg1:0x%.8x\n", readl(mdata->base + SPI_CFG1_REG));
    spi_debug("cfg2:0x%.8x\n", readl(mdata->base + SPI_CFG2_REG));
    spi_debug("cmd :0x%.8x\n", readl(mdata->base + SPI_CMD_REG));
    spi_debug("tx_s:0x%.8x\n", readl(mdata->base + SPI_TX_SRC_REG));
    spi_debug("rx_d:0x%.8x\n", readl(mdata->base + SPI_RX_DST_REG));
    spi_debug("status1:0x%.8x\n", readl(mdata->base + SPI_STATUS1_REG));
    spi_debug("pad_sel:0x%.8x\n", readl(mdata->base + SPI_PAD_SEL_REG));
    spi_debug("||*****%s end*****||\n", __func__);
}
```

Call `spi_dump_reg()` before invoking `mtk_spi_enable_transfer()` in both `mtk_spi_fifo_transfer()` and `mtk_spi_dma_transfer()` functions.

1.4.6 Seeking MTK Support

If you are still unable to resolve your issue after the troubleshooting steps mentioned earlier, and when seeking help from MTK, please also provide the relevant logs containing SPI register information, waveform charts, dts, and the output from `cat mt_gpio` as mentioned previously.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.